

COMPLETE LEARNING GUIDE

Deployment, Docker & DevOps

From Local Development to a Live, Maintainable Production System

WEEK 07 - DEPLOYMENT, DOCKER & DEVOPS

SDLC, Agile, environments, DNS and SSL, deployment, containers, CI/CD and production security.

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

Learn • Build • Connect • Grow

Course Information

Field	Detail
Program	Web Development Masterclass
Provider	Get Techy With Lucky / Tech Pulse Insider
Instructor	Lucky Nakola
Week	Week 07 of 08
Module	SDLC + Deployment + DevOps + Docker
Estimated study time	6-8 hours
Builds on	Week 06 - MySQL, Databases and Full-Stack Integration
Format	Read -> Practise -> Quiz -> Build -> Submit
Assessment	Weekly quiz (auto-scored) and a practical assignment submitted via GitHub

Learning Objectives

By the end of this week you should be able to:

1. Describe the phases of the software development life cycle and what each one produces.
2. Explain Agile, Scrum and Kanban, and when each is a sensible choice.
3. Distinguish development, staging and production, and configure an application per environment.
4. Explain how DNS, hosting, HTTPS and SSL certificates combine to serve a site at a domain.
5. Deploy both a static front end and a PHP and MySQL application to real hosting.
6. Explain what containers are, read a Dockerfile, and run a multi-service app with Compose.
7. Explain CI and CD, and set up a pipeline that checks every change automatically.
8. Apply production security basics and know what to do first when a deployment goes wrong.

Prerequisites

Before starting this week, make sure you can already do the following:

- Build a full-stack application with PHP and MySQL.
- Use Git and GitHub confidently, including branches and pull requests.
- Run commands in a terminal and read error output carefully.
- Understand what a client and a server are, and what HTTP does between them.

Table of Contents

This guide is written to be read in order. Each chapter builds on the one before it.

- 1.** Introduction - The Gap Between 'It Works' and 'It Is Live'
- 2.** The Software Development Life Cycle
- 3.** Agile, Scrum and Kanban in Practice
- 4.** Environments and Environment Variables
- 5.** How the Internet Delivers Your Site: DNS, Hosting and SSL
- 6.** Deploying a Front End
- 7.** Deploying a PHP and MySQL Application
- 8.** Containers and Docker
- 9.** DevOps and Continuous Integration
- 10.** Monitoring, Logging and Debugging in Production
- 11.** Important Terminology
- 12.** Common Mistakes and How to Avoid Them
- 13.** Security in Production
- 14.** Practical Exercise - Take a Project Live
- 15.** Knowledge Check - Weekly Quiz
- 16.** Assignment - Deploy, Document and Containerise
- 17.** Summary

Chapter 1 - Introduction - The Gap Between 'It Works' and 'It Is Live'

For six weeks the answer to 'does it work?' has been whether it runs on your machine. That answer stops being useful the moment somebody else needs to use what you built. Production has a different PHP version, no XAMPP, different file permissions, a case-sensitive filesystem, real users typing unexpected things, and no debugger attached.

This week is about closing that gap. Not just the mechanics of uploading files, but the professional practices around them: how software projects are organised, how configuration is separated from code, how a site is reached at a domain, how environments are made to match, and what you do at three in the morning when the live site is down.

WHY THIS MATTERS

'It works on my machine' is the most expensive sentence in software. Everything in this week - environment variables, staging, containers, CI - exists to make that sentence either true everywhere or impossible to say.

How to Use This Guide

Read this guide with an editor open beside it. Type every example yourself rather than copying and pasting. Typing is slower, and that is the point: it is what turns a concept you have read into a skill you own. When something does not work, resist the urge to look at the answer straight away. Read the error, form a guess about the cause, then test that guess. Debugging is the skill that separates people who can build from people who can only follow.

Chapter 2 - The Software Development Life Cycle

The SDLC is the sequence of stages a software project moves through. Different methodologies arrange and repeat these stages differently, but the stages themselves are always present - even when a team skips one, which is usually visible later in the bug count.

Phase	Question it answers	What it produces
Planning	Is this worth building, and can we?	Scope, budget, timeline, feasibility
Requirements analysis	What exactly must it do?	Functional and non-functional requirements
Design	How will it be built?	Architecture, database schema, UI designs
Implementation	Build it	Working code in version control
Testing	Does it do what we said?	Test results, a bug list, fixes
Deployment	Get it to users	A live, running system
Maintenance	Keep it working and improve it	Patches, monitoring, new features

NOTE

Maintenance is where most of a system's total cost is spent - commonly quoted at 60 to 80 percent. That is the real argument for readable code, useful commit messages and documentation: you are writing for the person who maintains this, who is usually you, later, having forgotten everything.

Waterfall Versus Iterative

Waterfall runs the phases once, in order, each completing before the next begins. It works when requirements genuinely cannot change - a regulated system, a fixed government contract. It fails badly when they can, because a mistake in requirements is only discovered at the very end when it is most expensive to fix.

Iterative approaches run the whole cycle repeatedly on small slices, shipping something usable each time. Most web work is iterative, because clients rarely know exactly what they want until they see something.

Chapter 3 - Agile, Scrum and Kanban in Practice

Agile is a set of values, not a process: working software over documentation, responding to change over following a plan, collaboration over contract negotiation. Scrum and Kanban are two concrete frameworks built on those values.

Scrum

Element	What it is
Sprint	A fixed time box, usually one to four weeks, producing something shippable
Product owner	Decides what gets built and in what order
Scrum master	Removes obstacles and protects the process
Development team	The people building it
Sprint planning	Choosing what the team commits to this sprint
Daily stand-up	Fifteen minutes: progress, plan, blockers
Sprint review	Demonstrating the finished work to stakeholders
Retrospective	Improving how the team works, not what it built

Kanban

Kanban has no sprints. Work is visualised on a board and pulled through columns, with a limit on how many items may sit in each. The limit is the whole point: it forces the team to finish things rather than start them. It suits support work and any flow of unpredictable, individually small tasks.

BACKLOG	TO DO (5)	IN PROGRESS (2)	REVIEW (2)	DONE
-----	-----	-----	-----	-----
Dark mode	Fix nav	Contact form	Card grid	Hero

Search | Add footer | | Setup

The numbers are work-in-progress limits. IN PROGRESS is full, so nobody may start anything new until something moves on.

A Kanban board. The constraint is the feature, not a limitation.

BEST PRACTICE

You are already doing a version of this. Your GitHub issues are a backlog, your feature branches are work in progress, and your pull requests are the review column. Making that explicit on a board is most of what adopting Kanban involves.

Chapter 4 - Environments and Environment Variables

	Development	Staging	Production
Who uses it	You	The team and the client	Real users
Data	Fake test data	A realistic copy	Real data
Errors	Shown in full on screen	Logged and shown	Logged only, never shown
Speed of change	Constant	Per release candidate	Deliberate and controlled
If it breaks	Nobody notices	The team notices	You have an incident

Configuration Belongs Outside the Code

The same codebase must run in all three environments. What differs is configuration: which database, which credentials, whether to display errors. Hard-coding those values means editing code to deploy, which means the thing you tested is not the thing you shipped.

```
// BAD - credentials in code, committed to Git, same everywhere
$db = new PDO('mysql:host=localhost;dbname=shop', 'root', 'password123');

// GOOD - supplied by the environment
$db = new PDO(
    'mysql:host=' . getenv('DB_HOST') . ';dbname=' . getenv('DB_NAME'),
    getenv('DB_USER'),
    getenv('DB_PASSWORD')
);
```

The code stops caring which environment it is in. That is the goal.

```
# .env -- local only, listed in .gitignore, NEVER committed
DB_HOST=localhost
DB_NAME=shop
DB_USER=root
DB_PASSWORD=password123
APP_ENV=development
APP_DEBUG=true
```

```
# .env.example -- committed, documents what is needed, holds nothing secret
DB_HOST=
DB_NAME=
DB_USER=
DB_PASSWORD=
APP_ENV=
APP_DEBUG=
```

Commit the shape of your configuration, never its values.

COMMON MISTAKE

Displaying errors in production is both a usability failure and a security hole. PHP error output routinely reveals file paths, database structure and sometimes credentials. Set `display_errors` off and `log_errors` on in production, always.

Chapter 5 - How the Internet Delivers Your Site: DNS, Hosting and SSL

What Happens When Somebody Types Your Domain

9. The browser checks its cache for the IP address of the domain.
10. If it does not have one, it asks a DNS resolver, which walks up to the authoritative name server.
11. DNS returns an IP address, cached for the length of the record's TTL.
12. The browser opens a TCP connection to that address, on port 443 for HTTPS.
13. A TLS handshake takes place: the server presents its certificate and the browser verifies it.
14. Once encrypted, the browser sends the HTTP request.
15. The server responds with HTML, and the browser requests the CSS, JS and images it references.

The DNS Records You Will Actually Set

Record	Purpose	Example
A	Point a domain at an IPv4 address	@ -> 203.0.113.10
AAAA	Point a domain at an IPv6 address	@ -> 2606:4700::1
CNAME	Alias one name to another name	www -> myapp.vercel.app
MX	Where email for this domain goes	@ -> mail.google.com
TXT	Arbitrary text, used for verification	Domain ownership proof

NOTE

DNS changes are not instant because answers are cached worldwide for the TTL you set. Lower the TTL a day before a planned migration and changes will propagate in minutes rather than hours. This one trick removes most of the anxiety from moving a live site.

HTTPS Is Not Optional

HTTPS does three things: it encrypts traffic so it cannot be read in transit, it guarantees the response was not altered on the way, and it proves the server genuinely controls the domain. Without it, any network between the user and your server - a cafe router, an ISP - can read and modify what you send.

Certificates are free from Let's Encrypt, and most hosting platforms now issue and renew them automatically. There is no remaining excuse for an unencrypted site, and browsers mark them as 'Not secure' precisely to make that clear.

Chapter 6 - Deploying a Front End

A site of HTML, CSS and JavaScript needs no server-side processing, so it can be served directly from a content delivery network. This is both the cheapest and the fastest option, and for most of what you have built so far it is the correct one.

Platform	Best for	Deploys from	Free tier
GitHub Pages	Portfolios and project sites	A GitHub repository	Yes
Netlify	Static sites with forms and functions	Git push	Yes
Vercel	Static sites and modern frameworks	Git push	Yes
Cloudflare Pages	Static sites needing a global edge	Git push	Yes

```
# Deploying to GitHub Pages
# 1. Push your site to a public repository
# 2. Settings -> Pages -> Source: main branch, / (root)
# 3. It goes live at https://username.github.io/repository-name/

# Deploying to Vercel or Netlify
# 1. Connect your GitHub account
# 2. Import the repository
# 3. Set the build command and output directory, e.g.
#     Build command:    npm run build
#     Output directory: dist
# 4. Every push to main now deploys automatically
```

Modern static deployment is connecting a repository once, then pushing.

COMMON MISTAKE

Two things break first-time static deployments. Absolute paths like /styles.css fail when the site is served from a subfolder - use relative paths. And Linux servers are case-sensitive, so Header.png and header.png are different files even though Windows treated them as the same.

Chapter 7 - Deploying a PHP and MySQL Application

A PHP application needs a server that can execute PHP and a MySQL database to talk to. That rules out static hosting and puts you on shared hosting, a VPS, or a platform that supports PHP directly.

16. Provision the hosting and note the PHP version. Confirm it matches what you developed against.
17. Create the production database and a dedicated user with only the privileges the app needs.
18. Export your local schema with mysqldump and import it into the production database.
19. Upload the application files, excluding .env, node_modules and anything in .gitignore.
20. Create the production .env on the server with the production credentials.
21. Point the web root at your public folder so source files are never directly reachable.
22. Turn display_errors off and log_errors on.
23. Enable HTTPS and force a redirect from http to https.
24. Test every critical path: registration, login, the main create-read-update-delete flows.
25. Set up an automated database backup before you announce the site to anyone.

```
# Export from local
mysqldump -u root -p shop_db > shop_db.sql

# Import into production
mysql -u shop_user -p shop_db_prod < shop_db.sql

# Grant only what the application needs - not ALL PRIVILEGES
GRANT SELECT, INSERT, UPDATE, DELETE ON shop_db_prod.* TO 'shop_user'@'localhost';
```

The production database user should not be able to drop tables. Least privilege applies here too.

COMMON MISTAKE

Never leave the application's source files inside the public web root. If PHP execution fails for any reason, the server will serve your source as plain text - including any credentials in it. Point the document root at a public/ folder and keep everything else above it.

Chapter 8 - Containers and Docker

The Problem Containers Solve

Your app needs PHP 8.2 with specific extensions, MySQL 8, and a particular Apache configuration. Reproducing that on a teammate's laptop, on staging and on production - identically - is the source of most 'works on my machine' failures. A container packages the application together with everything it needs to run, so the same package runs the same way everywhere.

	Virtual machine	Container
Contains	A full guest operating system	Just the app and its dependencies

	Virtual machine	Container
Size	Gigabytes	Tens or hundreds of megabytes
Start time	Minutes	Under a second
Isolation	Complete, hardware-level	Process-level, sharing the host kernel
Typical use	Running a different OS entirely	Packaging and shipping an application

The Three Ideas

- **Dockerfile.** A text file of instructions. The recipe.
- **Image.** The read-only result of building a Dockerfile. The cake.
- **Container.** A running instance of an image. The slice being eaten. You can run many from one image.

```
# Dockerfile - a PHP application on Apache
FROM php:8.2-apache

# Install the extensions the app needs
RUN docker-php-ext-install pdo pdo_mysql

# Apache should serve the public folder, not the project root
ENV APACHE_DOCUMENT_ROOT=/var/www/html/public
RUN sed -ri 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/sites-available/*.conf

# Copy the application in
COPY . /var/www/html

WORKDIR /var/www/html
EXPOSE 80
```

Note the pinned tag php:8.2-apache. Using :latest means your build changes without you changing anything.

Running More Than One Service

```
# docker-compose.yml
services:
  app:
    build: .
    ports:
      - "8080:80"          # localhost:8080 reaches port 80 in the container
    environment:
      DB_HOST: db
      DB_NAME: shop
      DB_USER: shop_user
      DB_PASSWORD: ${DB_PASSWORD}
    depends_on:
      - db

  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: shop
```

```
MYSQL_USER: shop_user
MYSQL_PASSWORD: ${DB_PASSWORD}
MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
volumes:
  - db_data:/var/lib/mysql # data survives container restarts

volumes:
  db_data:
```

One file describing the whole system. Note `DB_HOST` is 'db' - the service name is the hostname.

```
docker compose up -d # start everything in the background
docker compose ps    # what is running
docker compose logs -f app # follow the app's logs
docker compose down  # stop and remove containers (volumes survive)
docker compose down -v # also delete the volumes - destroys the database
```

The five Compose commands you need. Be careful with the last one.

WHY THIS MATTERS

The real win is onboarding. Without Compose, a new developer spends a morning installing PHP, MySQL and extensions and fighting version mismatches. With it, they clone the repository and run one command. That difference is worth the file.

Chapter 9 - DevOps and Continuous Integration

DevOps is a culture before it is a toolset: the people who build the software are also responsible for running it. The practical consequence is automation, because a team that owns deployment quickly stops wanting to do it by hand.

Term	What it means	In practice
Continuous integration	Every change is built and tested automatically	A workflow runs on every push
Continuous delivery	Every passing change is ready to release	A deployable artifact, released on a human's say-so
Continuous deployment	Every passing change is released automatically	Merging to main puts it live

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
```

```
- uses: actions/checkout@v4

- uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: 'npm'

- run: npm ci
- run: npm run lint
- run: npm run build
```

A complete CI workflow. Every pull request now proves it builds before anyone reviews it.

BEST PRACTICE

Start with one check that would actually have caught a real mistake you have made - usually 'does it build'. A pipeline nobody trusts because it is always red is worse than no pipeline.

Chapter 10 - Monitoring, Logging and Debugging in Production

In development you read errors on the screen. In production, showing an error to a user is a bug in itself, so errors go to a log instead - and something has to be watching.

- **Log what happened, never what is secret.** Log that a login failed and for which account; never log the password that was tried.
- **Use levels.** Debug, info, warning, error. In production, record warnings and errors and sample the rest.
- **Monitor from outside.** An uptime service checking every minute tells you the site is down before a customer does.
- **Watch the error rate, not just uptime.** A site that responds with 500 on every request is technically up.
- **Back up, and test the restore.** A backup you have never restored is a hypothesis, not a backup.

When Production Breaks

26. Restore service first. Roll back to the last known-good version.
27. Confirm the rollback actually worked - check the site and the error rate, do not assume.
28. Communicate. Tell the client or the users something, even if it is only that you are on it.
29. Only now, diagnose. Reproduce the failure in staging using the production logs as evidence.
30. Fix it properly on a branch, with a test or a check that would have caught it.
31. Write down what happened and what you changed so it does not happen twice.

COMMON MISTAKE

The instinct to debug directly on the live server while users are affected is strong and almost always wrong. Every minute spent investigating is a minute the site stays broken, and editing files on production means the fix exists nowhere in version control.

Chapter 11 - Important Terminology

This is the vocabulary of professional practice. It is what interviews probe and what job descriptions assume, so being able to use these terms precisely has direct career value.

Term	Definition	In Plain Words	Example / Use Case
SDLC	The Software Development Life Cycle - the stages a software project moves through from idea to maintenance.	The map of how software gets built.	Planning, analysis, design, build, test, deploy, maintain
Requirements	A recorded statement of what the software must do.	What the client actually asked for.	'Users must reset their own password'
Functional requirement	Something the system must do.	A feature.	'The system sends a confirmation email'
Non-functional requirement	A quality the system must have.	How well it must do it.	'Pages load in under two seconds on 3G'
Agile	An approach delivering software in short iterations with frequent feedback.	Build a bit, show it, adjust.	Two-week sprints
Waterfall	A sequential approach where each phase completes before the next begins.	Finish one stage, then start the next.	Common in fixed-scope government contracts
Scrum	An Agile framework organised around sprints, roles and fixed ceremonies.	A structured way of doing Agile.	Sprint planning, daily stand-up, retrospective
Sprint	A fixed time box in which a set amount of work is completed.	A short delivery cycle.	Two weeks
Kanban	A method visualising work on a board and limiting work in progress.	A board of cards you pull from.	To Do / In Progress / Done
Stand-up	A short daily meeting covering progress, plans and blockers.	A quick daily sync.	Fifteen minutes, standing up
Retrospective	A meeting reviewing how the team worked, not what it built.	What should we change next time?	Held at the end of a sprint
Backlog	The ordered list of work not yet done.	The queue of everything outstanding.	Prioritised by the product owner
User story	A requirement written from the user's point of view.	A feature described by who wants it and why.	'As a customer I want to track my order'
Deployment	Making a version of the software available in an environment.	Putting it live.	Pushing the site to production
Environment	A configured place where the application runs.	One of the copies of your app.	Development, staging, production
Development environment	Your local machine, set up for building and debugging.	Where you write code.	XAMPP on your laptop
Staging environment	A production-like environment used for final checks.	A dress rehearsal.	staging.example.com
Production	The live environment real users	The real thing.	example.com

Term	Definition	In Plain Words	Example / Use Case
	depend on.		
Environment variable	A configuration value supplied to the app from outside its code.	A setting that changes per environment.	DB_HOST, API_KEY
.env file	A local file holding environment variables, never committed.	Where your local secrets live.	Listed in .gitignore
Configuration drift	Environments quietly diverging in setup over time.	'It works on staging but not production.'	A PHP version upgraded on one server only
Hosting	A server that stores your files and serves them to visitors.	Where your site lives.	Shared hosting, a VPS, or a cloud platform
Shared hosting	One server hosting many customers' sites.	Cheapest option, least control.	cPanel hosting
VPS	A virtual private server - your own isolated slice of a machine.	Your own server, virtually.	A DigitalOcean droplet
Static hosting	Hosting for files needing no server-side processing.	For HTML, CSS and JS only.	GitHub Pages, Netlify, Vercel
Domain name	The human-readable address of a site.	The name people type.	gettechy.co.ke
DNS	The system translating domain names into IP addresses.	The internet's phone book.	Resolving example.com to 93.184.216.34
A record	A DNS record pointing a domain at an IPv4 address.	'This name lives at this IP.'	@ -> 203.0.113.10
CNAME record	A DNS record pointing one name at another name.	'This name is an alias for that one.'	www -> myapp.vercel.app
TTL	How long a DNS record may be cached before it is looked up again.	How stale an answer may get.	3600 seconds
Propagation	The delay while caches worldwide pick up a changed DNS record.	Why a domain change is not instant.	Usually minutes, sometimes a day
SSL/TLS	The encryption protocol securing traffic between browser and server.	The lock in the address bar.	Enables HTTPS
SSL certificate	A file proving a site owns its domain, enabling HTTPS.	The site's identity document.	Issued free by Let's Encrypt
HTTPS	HTTP carried over an encrypted TLS connection.	The secure version of HTTP.	https://example.com
Container	An isolated, packaged environment running an application with its dependencies.	Your app plus everything it needs, boxed up.	A Docker container
Docker	The most widely used tool for building and running containers.	The container tool.	docker run, docker build
Image	A read-only template a container is started from.	The recipe.	php:8.2-apache
Dockerfile	A text file of instructions for building an image.	The build script for your box.	FROM, COPY, RUN, CMD
Docker Compose	A tool for defining and running multi-container applications.	Start your app and its database together.	docker-compose.yml

Term	Definition	In Plain Words	Example / Use Case
Volume	Storage that persists beyond a container's lifetime.	Where data survives a restart.	Keeping MySQL data between runs
Port mapping	Connecting a port on the host to a port inside a container.	How you reach the app in the box.	-p 8080:80
DevOps	A culture and set of practices uniting development and operations.	Building and running the software as one team.	Developers own deployment too
CI (continuous integration)	Automatically building and testing every change as it is pushed.	A robot checks your work immediately.	GitHub Actions on every push
CD (continuous deployment)	Automatically releasing changes that pass the pipeline.	Merged means live.	Vercel deploying on merge to main
Pipeline	The automated sequence of steps a change passes through.	The assembly line.	Lint, test, build, deploy
Build artifact	The output of a build that gets deployed.	The finished package.	The dist folder
Rollback	Returning to a previous working version after a bad release.	Undo the deployment.	Redeploying the last good build
Zero-downtime deployment	Releasing without the site becoming unavailable.	Users never see it happen.	Switching traffic once the new version is ready
Logging	Recording what the application did, for later inspection.	The app's diary.	Error logs on the server
Monitoring	Watching a running system's health and behaviour.	Knowing it is up before users tell you.	Uptime checks and alerts
Uptime	The proportion of time a service is available.	How reliable it is.	99.9% uptime
Backup	A copy of data kept so it can be restored after loss.	Insurance for your data.	A nightly database dump
Scalability	The ability to handle growth in load.	Coping when it gets busy.	Adding servers as traffic grows

Chapter 12 - Common Mistakes and How to Avoid Them

Mistake	What goes wrong	The fix
Credentials committed to Git	Permanently exposed; bots find them in minutes	Use .env and .gitignore from the first commit
display_errors on in production	Leaks paths, schema and sometimes secrets to users	Log errors; never display them
Source files inside the web root	Served as plain text if PHP fails	Point the document root at public/
Using :latest for a base image	Builds change without you changing anything	Pin an explicit version tag
No database backup	One mistake destroys everything irrecoverably	Automate backups and test a restore

Mistake	What goes wrong	The fix
Deploying straight to production	Users are your test environment	Add a staging environment that mirrors production
Absolute asset paths	Assets 404 when served from a subfolder	Use relative paths
Ignoring filesystem case sensitivity	Works on Windows, 404s on Linux	Keep filenames lowercase and consistent
Manual deployment steps in someone's head	Nobody else can deploy or roll back	Write DEPLOYMENT.md with exact commands
Debugging on the live server	Extends the outage; fixes exist nowhere in Git	Roll back first, diagnose in staging

Chapter 13 - Security in Production

- **HTTPS everywhere, with http redirected to https.** Certificates are free and automatic; there is no reason not to.
- **Least privilege on the database.** The application user needs SELECT, INSERT, UPDATE and DELETE. It does not need DROP.
- **Keep dependencies patched.** Most real breaches exploit a known vulnerability in an old library, not clever new attacks.
- **Never trust input, at any layer.** Frontend validation is a courtesy to users; backend validation is the actual defence.
- **Use prepared statements without exception.** This is what stops SQL injection, and it costs nothing.
- **Hash passwords with a modern algorithm.** `password_hash()` in PHP. Never store or log a plaintext password.
- **Set security headers.** Content-Security-Policy, X-Content-Type-Options and Strict-Transport-Security are cheap and effective.
- **Rate-limit authentication endpoints.** Without a limit, a login form is a free brute-force target.
- **Two-factor authentication on every account that can deploy.** Your hosting and GitHub accounts are the keys to everything.

NOTE

Security is layered. No single control is sufficient, and the goal is not perfection - it is making yourself a harder target than the automated scanners that will find your site within hours of it going live.

Chapter 14 - Practical Exercise - Take a Project Live

32. Take your Week 3 Tailwind site and deploy it to GitHub Pages, Netlify or Vercel.

33. Confirm it loads over HTTPS and that no asset returns 404. Fix any absolute or mis-cased paths.
34. Add a .env.example to your Week 6 PHP project listing every variable it needs.
35. Refactor that project so every credential comes from getenv() instead of being hard-coded.
36. Write a Dockerfile for it, pinning an explicit PHP version.
37. Build the image and run it. Fix whatever breaks - something will.
38. Write a docker-compose.yml adding a MySQL service with a named volume.
39. Run docker compose up and confirm the app connects to the database in the container.
40. Stop everything with docker compose down, start it again, and confirm the data survived.
41. Add a GitHub Actions workflow that runs on every push and fails on a broken build.
42. Deliberately break the build, push it, and watch CI catch it. Then fix it.
43. Write DEPLOYMENT.md with exact deploy and rollback commands.
44. Hand your repository to a classmate and ask them to run it using only your documentation. Fix whatever they had to ask you about.

Chapter 15 - Knowledge Check - Weekly Quiz

This week's questions are as much about judgement as recall. The short-answer questions matter most - they are the ones that come up in interviews.

Pass mark: 70% | *Answers are at the end of this chapter.*

Q1. Which SDLC phase produces the answer to 'what exactly must this system do?'

- a) Design
- b) Requirements analysis
- c) Implementation
- d) Maintenance

Q2. Why must a database password be supplied as an environment variable rather than written in the code?

- a) It runs faster that way
- b) It keeps the secret out of version control and lets each environment use a different value
- c) PHP cannot read strings from files
- d) It is required by HTTPS

Q3. What does a Dockerfile describe?

- a) A running container
- b) The instructions for building an image
- c) The database schema
- d) The DNS configuration

Q4. Which DNS record type points a subdomain at another hostname?

- a) A record
- b) MX record
- c) CNAME record
- d) TXT record

Q5. Docker containers each run their own full operating system, like virtual machines.

True / False

Q6. If the code works on your laptop, it will work in production.

True / False

Q7. Continuous integration means automatically deploying every commit to production.

True / False

Q8. Explain the difference between development, staging and production environments, and why a team keeps all three.

Short answer - write two or three sentences.

Q9. Your site works on http:// but you are told to enable HTTPS. Explain what HTTPS actually does and why it is required even for a site with no login.

Short answer - write two or three sentences.

Q10. A deployment has gone wrong and the live site is broken. Describe your first three actions, in order.

Short answer - write two or three sentences.

Answer Key

Mark your own work honestly - the goal is to find your gaps, not to score well.

#	Answer	Why
Q1	b - Requirements analysis	Requirements come before design. Building the wrong thing correctly is the most expensive mistake in software, and it always traces back to skipped requirements work.
Q2	b - Keeps the secret out of version control and varies per environment	Two problems solved at once: the credential never enters your Git history, and development, staging and production can each point at their own database without a code change.
Q3	b - The instructions for building an image	A Dockerfile is the recipe. Building it produces an image, and running the image produces a container. Recipe, then cake, then a slice being eaten.
Q4	c - CNAME record	An A record points at an IP address; a CNAME points at another name. Platforms like Vercel and Netlify give you a hostname, so you use a CNAME for www.
Q5	False	Containers share the host kernel and isolate only the process and filesystem. That is why a container starts in under a second while a virtual machine takes minutes.
Q6	False	Different PHP versions, missing extensions, different file permissions, absent environment variables and case-sensitive filesystems all break this assumption. Containers exist largely to close that gap.
Q7	False	That is continuous deployment. Continuous integration is the automated building and testing of every change; whether it then ships automatically is a separate decision.
Q8	Development is the	The reasoning matters more than the names: each environment trades

#	Answer	Why
	developer's local machine, optimised for fast iteration and full of debug output. Staging mirrors production as closely as possible and is where changes get a final check against realistic data. Production is what real users depend on. Three environments exist so risky work never happens where it can hurt real people.	speed against safety differently.
Q9	HTTPS encrypts traffic between browser and server so it cannot be read or altered in transit, and proves the server really is the domain it claims. Even without a login it prevents networks injecting adverts or malicious code, avoids browser 'Not secure' warnings, and is required for search ranking and many browser features.	Students often think HTTPS is only about passwords. Integrity and authenticity matter on every site.
Q10	First, roll back to the last known-good version so users are served a working site. Second, confirm the rollback worked by checking the site and the error logs. Third, reproduce the failure in staging and diagnose it there, away from production.	Restore service first, diagnose second. Debugging in production while users are affected is the classic beginner instinct and the wrong one.

Chapter 16 - Assignment - Deploy, Document and Containerise

Objective

Take one of your existing projects from local development to a documented, secured, publicly accessible deployment, and containerise it so it runs identically on any machine.

Scenario

The business you have been building for since Week 2 is ready to go live. They need a real URL, HTTPS, and confidence that the site can be rebuilt if the server is lost. Their next developer must be able to get the project running locally in under ten minutes.

Requirements

- Deploy your front-end project to a real, publicly reachable URL.
- Serve it over HTTPS with a valid certificate.
- Move every configurable value out of the code and into environment variables.
- Commit a .env.example documenting each variable, with no real values.
- Write a Dockerfile that builds and runs the project.
- Write a docker-compose.yml bringing up the application together with a database service.
- Set up a CI workflow that runs on every push to main.
- Write a DEPLOYMENT.md covering how to deploy, how to roll back, and where the logs are.
- Document the DNS records required if a custom domain were attached.

Expected Output

A working public URL served over HTTPS, plus a repository that a stranger can clone and run with a single docker compose up command, with documentation good enough that they never need to ask you a question.

Technical Requirements

- No credentials of any kind committed to the repository at any point in its history.
- The Dockerfile uses a specific base image tag, never latest.
- docker-compose.yml defines at least two services and a named volume for database persistence.
- The CI workflow fails the build if the project does not build cleanly.
- DEPLOYMENT.md includes the exact commands, not prose descriptions of them.
- The README links to the live site and states which platform hosts it.

Submission Instructions

45. Push everything to a public GitHub repository named week-07-deployment.

46. Put the live URL at the top of the README.
47. Include a screenshot of your CI workflow passing.
48. Submit the repository link and the live URL on the Week 7 assignment page.

Evaluation Criteria

Criterion	What Earns Full Marks	Weight
Live deployment	Site is publicly reachable over HTTPS and actually works	25%
Configuration handling	Nothing hard-coded; .env.example complete; no secrets in history	20%
Containerisation	Dockerfile and Compose file build and run cleanly from scratch	20%
CI pipeline	Workflow runs automatically and genuinely catches a broken build	15%
Documentation	DEPLOYMENT.md lets a stranger deploy and roll back unaided	15%
Professional practice	Sensible commits, clean repository, clear README	5%

Bonus Challenges

- Attach a real custom domain and configure the DNS records yourself.
- Add a staging environment that deploys automatically from a develop branch.
- Add an automated database backup script and document the restore procedure.
- Add an uptime monitor and include a screenshot of it in the README.

Chapter 17 - Summary

- The SDLC phases always happen; methodologies differ in how often they repeat and in what order.
- Agile is a set of values. Scrum adds sprints and ceremonies; Kanban adds a board and work-in-progress limits.
- Development, staging and production exist so risky work never happens where real users are.
- Configuration lives in environment variables, never in code, and secrets never enter Git.
- DNS maps a name to an address, hosting serves the files, and TLS certificates make the connection trustworthy.
- Static front ends belong on a CDN platform; PHP applications need a server that executes PHP.
- A Dockerfile is a recipe, an image is the result, a container is a running instance. Compose runs several together.
- CI builds and tests every change; CD releases it. Start with one check you would actually trust.
- In production, errors are logged and never displayed, backups are tested, and monitoring watches from outside.

- When production breaks: restore service, confirm, communicate, then diagnose in staging.

You are not expected to remember everything in this guide. Professional developers look things up constantly. What you are expected to build is a mental model: knowing that a thing exists, roughly how it behaves, and where to look it up when you need the detail.

Further Reading and References

Documentation changes faster than any printed guide. Learning to read the official docs is part of the skill you are building.

- **MDN: What is a web server?** - https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_is_a_web_server
- **Docker documentation - Get started** - <https://docs.docker.com/get-started/>
- **Docker Compose reference** - <https://docs.docker.com/compose/>
- **GitHub Actions documentation** - <https://docs.github.com/en/actions>
- **Let's Encrypt - free certificates** - <https://letsencrypt.org/>
- **The Twelve-Factor App - config and environments** - <https://12factor.net/>
- **OWASP Top Ten - the essential security checklist** - <https://owasp.org/www-project-top-ten/>
- **Cloudflare Learning Center - DNS explained** - <https://www.cloudflare.com/learning/dns/what-is-dns/>
- **Agile Manifesto** - <https://agilemanifesto.org/>